ESCAPE ROOM · JOGOS MORTAIS · JIGSAW OS

Contexto técnico do projeto

Gerado de CLAUDE.md em 11/11/2026

Leia este arquivo antes de começar. Ele resume o projeto e onde paramos.

O projeto

Evento de escape room com tema Jogos Mortais do **CA de Engenharia da Computação**: grupos de até 5 pessoas, 30 min por grupo e um vilão que fala por câmera e projetor. Este repositório é o **jogo no notebook da sala**: uma simulação de computador (área de trabalho) com os minigames dentro.

- HTML/CSS/JS puro, sem build e sem internet. Para rodar, basta abrir `index.html` no Chrome.
- Idioma da interface: português (BR). Tom: terror psicológico, sem gore.
- Preferências do usuário: fazer perguntas quando faltar informação; ser econômico, mas sem perder qualidade.
- Branch de trabalho: `claude/relaxed-planck-j4r121`.

Estrutura

Arquivo	O que é
`index.html`	Telas: escolha do nível (se não houver `?nivel=`) → ligar → boot → (login opcional) → área de trabalho. Ctrl+Shift+1/2/3 trocam de nível
`js/config.js`	Configuração geral: nível, boot, sustos (`scareMedia`, `scareRules`, volume), sons
`js/os.js`	"Sistema operacional": janelas, ícones, diálogos (`OS.dialog`), notificações, `OS.showIcon`, `GATE`, revelar senha (`api.win`)

Arquivo	O que é
`js/interference.js`	Sustos: `Interference.scare(lugar)` (vídeo/imagem do lugar ou a máscara SVG + BRAAAM), `Interference.error(lugar)` (erro: só às vezes vira susto), `Interference.run()` (interferência do hacker, Ctrl+Shift+H). **A entrada do susto é sorteada** (`scareRules.entries` , nunca a mesma duas vezes seguidas): `seco` (sem aviso), `puxada` (preto + `riser`), `silencio` (preto + coração batendo por tempo incerto), `alivio` (preto, a tela volta ao normal e aí vem o golpe). O golpe (CSS `sc-*`): a máscara já nasce grande e avança, treme, dá espasmos, entorta em quadros, rasga em fatias, separa as cores e pisca, com a mandíbula caindo (`jf-jaw`); termina num corte seco para o preto. Às vezes vem uma **réplica** (`Interference.aftershock` , `scareRules.aftershockChance`): a máscara pisca de novo 1 a 3 s depois. **A máscara** (`Interference.mask()` , usada também no salto da cena final) foi redesenhada: sombra, cabelo, órbitas fundas escorrendo, nariz de caveira, boca escancarada com dentes; o zoom mantém o rosto inteiro legível (testado: zoom maior vira borrão e assusta menos). **Som da máscara:** `Interference.maskHit()` **, o "BRAAAM"** feito a partir da serra do "VOCÊ ERROU" (o som que mais assustou nos testes), uma quinta acima (165 Hz), com segunda menor e trítono, distorção, rosnado de 40 Hz e estrondo de prato, com `Interference.hiss()` (chiado baixo que abaixa no golpe). **Não trocar esse som sem pedir:** histórico do usuário: guincho estridente (desconfortável demais) → impacto grave (ruim) → stinger de orquestra ("não assustador") → grito sintetizado ("esquisito") → BRAAAM. Para uma gravação real, use `sounds.scare` . `Interference.shriek()` ficou só para o "rasgo". Efeitos leves dos erros (`minor`): glitch, sussurro, clarão, rasgo e **espiada** (a máscara aparece calada num canto e some). `CONFIG.scareStrobe = false` tira o clarão piscando
`js/phaselog.js`	Registro de tempo por etapa (Ctrl+Shift+L)
`js/doc.js`	`Doc.html(blocos)` e `Doc.paras(linhas)` : textos formatados dos 3 níveis (título, passos, alerta, destaque, assinatura; `código` e **negrito**). Os recados do J (`intro` , `caso`) ficam em blocos nos dados de cada nível; e-mails usam `Doc.paras`
`js/level1-data.js` / `js/level1.js`	**Nível 1 "As Notas"**: dados do caso / apps do desktop
`js/level2-data.js` / `js/level2.js`	**Nível 2 "O Cúmplice"**: dados do caso / apps do desktop
`js/level3-data.js` / `js/level3.js`	**Nível 3 "A Prova Final"**: dados do caso / apps do desktop
`js/minic.js`	Mini interpretador de C (`MiniC.compile` / `MiniC.runtime`) usado pelo `decifra.c` do Nível 3: erros de compilação com linha, segfault, lixo de memória, `%` do C
`projetor.html`	Tela do projetor no PC do mestre do jogo, para os 3 níveis (cronômetro, punição P, dicas D, vitória V)

Arquivo	O que é
`js/final-scare.js`	Cena do susto da resposta final errada (**P nos 3 níveis**; `FinalScare.play({ errs, fakeEnd })`, com as caixas de erro de cada nível): parabéns → "você errou" + luz vermelha → preto → cmd → chuva de erros (7 s, desacelera no fim) → transição de ~4,6 s (o som do sistema "desliga" descendo de tom, o alarme some, o cmd se apaga, as bordas escurecem, entra um zumbido grave bem baixo; os erros começam a voar em órbita em volta do centro, balançando, um depois do outro numa onda que dá a volta, e aos poucos se acomodam formando o círculo; a fila reta foi descartada porque o usuário achou esquisita) → daqui até o fim erros novos não param de surgir, sempre fora do rosto e cada vez mais rápido (pedido do usuário: confundir a cabeça antes do susto) → dois olhos de neon vermelho com pontinho branco (~5 s, sem sobrancelha) → sorriso → **final sorteado** (`CONFIG.finalScareEndings`, sem repetir o último): `bote` (puxada curta e a máscara salta no meio dos erros), `apagao` (tudo some, preto e silêncio por 2 a 4 s, e ela salta do escuro) ou `falso_fim` (a cena "acaba", o projetor mostra a mensagem de resposta errada e a máscara salta por cima). O salto usa a máscara nova (`Interference.mask()`), para um pouco antes de sair da tela (olhos e boca visíveis) e toca o BRAAAM (cerca de 30 s no total). Tudo HTML/CSS + WebAudio (o vídeo do Gemini não ficou bom)
`imprimir-nivel1.html` / `-nivel2` / `-nivel3`	Papéis da sala de cada nível
`TESTES-NIVEL1.md` / `-NIVEL2` / `-NIVEL3`	Guias (spoiler): respostas, lógica, dicas prontas. **Atualize junto com os dados do nível**
`GUIA-MESTRE.md` / `DICAS-MESTRE.md`	Guia passo a passo dos 3 níveis e banco de dicas por situação, para o mestre do jogo (PDFs `Guia-do-Mestre.pdf` e `Dicas-do-Mestre.pdf`). **Atualize junto com os dados dos níveis**
`gerar-pdfs.py` / `pdf/`	Gera os PDFs dos `.md` (Markdown → HTML estilizado → Chrome sem janela imprime em A4). **Rode de novo sempre que mudar um** `.md`
`videos/`	Vídeos dos jump scares (listar em `CONFIG.scareMedia`)
`apps/apps.js`, `apps/terminal.js`, `apps/wires.js`	Lista base `APPS` (cada nível a substitui); o terminal hacker e o painel de fios são opcionais do Nível 3

Atalhos do mestre do jogo no notebook: **Ctrl+Shift+X** reinicia (mesmo nível), **Ctrl+Shift+1/2/3** troca de nível, **Ctrl+Shift+H** dispara a interferência, **Ctrl+Shift+L** mostra o tempo das etapas. Não use Ctrl+Shift+N/T/W: o Chrome reserva esses atalhos e a página não os recebe.

Os três níveis seguem o mesmo formato: um mistério só, pistas no PC + na sala, minijogos opcionais com pistas extras, resposta **falada** ao mestre do jogo (que é o J), cronômetro/punição/vitória no `projetor.html`, dicas 1 por jogador pela câmera. `level1.js`, `level2.js` e `level3.js` só rodam no seu nível (`CONFIG.level`, de `?nivel=`) e cada um troca a lista `APPS`.

Nível 1 — "As Notas" (`index.html?nivel=1`)

Para iniciantes (mais guiado que o Nível 2). O J trocou a senha do Sistema de Notas do **Prof. Muriel Godoi**; quando o tempo acabar, as notas são zeradas. Detalhes e respostas em `TESTES-NIVEL1.md`.

- **Só o nome do professor é real** (decisão do usuário); filha, formatura, número da sorte, fotos e e-mails são inventados. **O professor ainda não autorizou** — o nome fica só em `L1.professor`.

- Resposta: **ALICE** (palavra única, decisão do usuário: os 3 níveis terminam com "A palavra-chave é X"). O "Esqueci a senha" tem **perguntas de segurança** (`L1.security`: ano da formatura 1994+4 = 1998 e número da sorte 111 em binário = 7, em decimal); certas, mostram o lembrete "o nome da minha filha". Resposta errada = tecla P no projetor (susto + perde **2 min**).
- Trilha principal: Sistema de Notas → E-mail da escola (anexo turma 3B) + desenhos na sala → `formatura.c` (lacuna + diploma na sala) → post-it 111 + tabela binária / `aula_binario`.
- Opcionais: `anotacoes.docx` (texto branco, Ctrl+A), `memoria.exe` (libera a foto da formatura), Navegador (histórico), Lixeira (isca + armadilha `NAO_ABRA.exe`).
- O Nível 1 antigo (admin, hacker, labirinto, cmd, ransomware) foi removido a pedido do usuário; está na tag git `nivel1-antigo`.

Nível 2 — "O Cúmplice" (`index.html?nivel=2`)

Partida separada de 30 min (cada grupo escolhe o nível na inscrição). Mesmo hacker (J), história nova: descobrir quem instalou o vírus no Lab. N112. Detalhes e respostas em `TESTES-NIVEL2.md`.

- Resposta: **Elisa**, falada ao mestre do jogo. Acusação errada = tecla P no `projetor.html` (cena do `final-scare.js` + perde 5 min + segunda chance). Dicas: 1 por jogador, pedidas pela câmera.
- Trilha principal: senha do notebook (cartas na sala) → `fechadura.exe` (deduzir AND/OR pelo registro) → lista de monitores (sala) → `chat_monitores` (senha = saída do `limpeza.c`) → depoimentos + bilhete de ônibus.
- Opcionais com pistas extras: `terminal.exe` (arquivos ocultos), `cameras.exe` (horário), `email_bruno` (senha deduzida), `rede.exe` (MAC decimal → hex). Erros = jump scare.
- O `level1.js` e o `level2.js` só rodam no seu nível (`CONFIG.level`, vindo de `?nivel=`). O Nível 2 troca a lista `APPS`.
- Decisões do usuário: só um notebook na sala; projetor ligado ao PC do mestre do jogo; sem reviravolta no final; sala real ainda não definida.

Nível 3 — "A Prova Final" (`index.html?nivel=3`)

O mais difícil. O J se revela: ex-aluno de Eng. de Computação que trancou no 3º semestre por causa de um segfault. O vírus apaga tudo quando o tempo acabar; o kill switch está numa mensagem cifrada. Detalhes e respostas em `TESTES-NIVEL3.md`.

- **Decisão do usuário:** as questões principais exigem conhecimento do curso (só **C + estruturas de dados** e **circuitos digitais** podem ser cobrados sem explicação; álgebra linear, cálculo e física não). Cada principal tem um **opcional sem conhecimento prévio** que dá uma **ajuda** para ela (guardada em `ajudas.txt`). O que é de fora do curso (ASCII, Morse) vem explicado nos papéis.
- Resposta: **SEGFAULT**. Errar = P no projetor (cena do `final-scare.js` + perde **5 min**).
- Principais: login (dump hex → ASCII = PONTEIRO) → em paralelo: lista encadeada com cartões na sala (MALLOC), `display.exe` (expressão do segmento g, ≤ 12 literais → DEAD), registrador de flip-flops D na sala (5 pulsos → 110 = msg_6) → `decifra.c` (só a função é editável; 5 bugs reais; roda no `js/minic.js`) com `msg_6` + `MALLOCDEAD`.
- Os 8 `msg_N` com a chave certa dizem "O KILL SWITCH E ..." com palavras diferentes: sem o registrador não dá para chutar.

- Opcionais → ajuda (o usuário pediu que fossem **difíceis**, mas sem exigir conhecimento): `radio.exe` (frase longa em Morse rápido) → lista; `placa.exe` (quebra-cabeça de girar as trilhas de uma placa 6×6 até ligar tudo ao chip; o J gira 2 peças a cada 50 s; substituiu o jogo da memória, que o usuário achou chato) → Karnaugh; `energia.exe` (`WiresGame` com `options`: pinos só com número, manual num papel na sala, curto desliga tudo) → registrador; `invasao.exe` (`TerminalGame` com `options`: sem dicas, rastreamento de 75 s) → bugs do C. Lixeira: armadilha `kill_switch.exe`.
- Decisões do usuário: história do J ex-aluno; edição só da função; questões em paralelo + C no final.

Pendências / decisões em aberto

- **Autorização do Prof. Muriel Godoi** para usar o nome no Nível 1.
- **Sala real do evento**: ainda não decidida (o Nível 2 usa "N112" em `js/level2-data.js`).
- **Vídeos dos sustos**: o usuário vai gerar com IA. Cada lugar tem sua lista em `CONFIG.scareMedia` (arquivos em `videos/`); com vários, sorteia sem repetir. Lugares: padrao, interferencia, nivel1, nivel1_armadilha, nivel2, nivel2_armadilha, nivel3, nivel3_armadilha, projetor_tempo (o `projetor_punicao` foi removido: a resposta errada toca a cena do `final-scare.js` nos 3 níveis).
- **Sustos por erro são condicionais** (`Interference.error` + `CONFIG.scareRules`): nunca no 1º erro, descanso de 90 s após qualquer susto, chance crescente (30% por erro). Sem susto = efeito leve. Armadilhas e projetor sempre assustam. **Aviso ao público:** os sustos têm luz piscando e som alto; vale um aviso na entrada.
- **Sons**: ainda sem arquivos (pasta `sounds/` + `CONFIG.sounds.*`).
- **Nível 3**: implementado, ainda **não testado com jogadores** (tempo e dificuldade a calibrar).

Como testar

Abra `index.html` e escolha o nível. No console do navegador, `CONFIG.bootSpeed = 20` acelera o boot; `OS.openApp('id')` abre qualquer app. Para testar localmente com servidor: `python -m http.server 8123` na pasta do projeto (o navegador pode guardar JS antigo em cache; recarregue com Ctrl+F5).